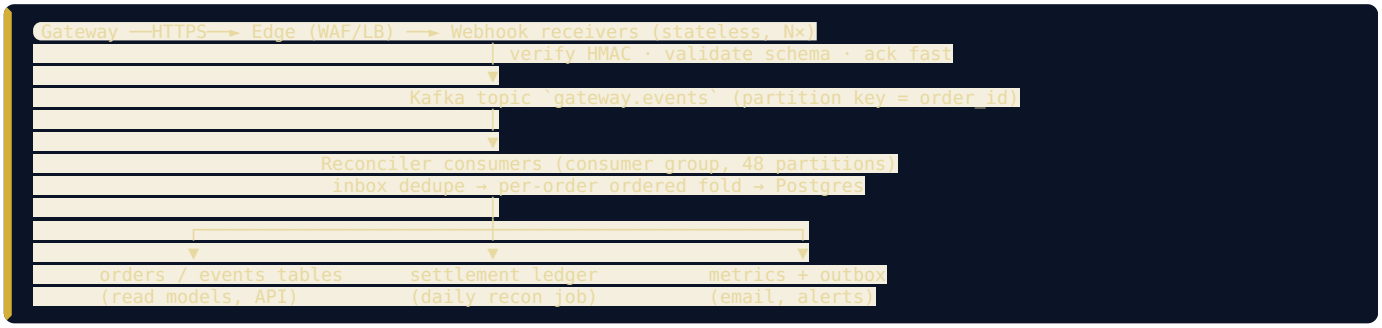
Production Design Note — Live Webhook Intake at 2,000 Events/Second

Scope: run the reconciler as a live webhook service for Gharana Home Co., ingest **2,000 events/sec sustained** (≈172.8M/day, peak ~6,000/sec), with correct money, no lost or double-applied events, and Finance-grade drift alarms.

1. Shape of the system



Receivers do **verify → validate → persist to Kafka → 202** in under 20 ms. All heavy work is asynchronous; the gateway never waits on our database. `order_id` as partition key guarantees all events for one order land on one consumer in order — the property that makes the state machine trivially correct without locks across the fleet.

2. Idempotency (exactly-once effect)

- Gateway delivers **at least once**. Every event carries a unique `event_id` — treat it as the idempotency key.
- Consumer inbox pattern: `INSERT INTO processed_events(event_id) … ON CONFLICT DO NOTHING` **in the same transaction** as the state mutation. Zero rows ⇒ duplicate ⇒ ack and drop. Unique index makes this enforceable even under rebalances and replays.
- Belt and braces: Kafka `enable.idempotence=true` on producers; consumers are at-least-once but the inbox makes the effect exactly-once. Duplicates metric (`webhook_duplicates_total`) is a first-class alert — a sudden spike means the gateway is retrying us.

3. Concurrency — two events for one order at the same moment

- **Partition ordering** (§1): same `order_id` ⇒ same partition ⇒ same consumer ⇒ sequential fold. No cross-worker race on an order, by construction.
- Inside the consumer, per order: `SELECT … FROM orders WHERE order_id = ? FOR UPDATE` (or a `pg_advisory_xact_lock(hashtext(order_id))`), append event, **recompute** state from the order's event log ordered by `sent_at, event_id` , write back with `version = version + 1` (optimistic guard on the read model too).
- Cross-order parallelism is unlimited; per-order work is a few row writes. At 2,000/sec and ~3 single-digit ms per event, a 12–16 consumer deployment has >5× headroom; scale by partitions + consumers together.
- Reordering across partitions never happens for one order; replay tooling re-publishes into the same key.

4. Database design (PostgreSQL 16)

```
CREATE TABLE orders (
  order_id        text PRIMARY KEY,
  customer_id     text NOT NULL,
  amount_paise    bigint NOT NULL CHECK (amount_paise >= 0),   -- integer paise, never float
  status          text NOT NULL DEFAULT 'CREATED',
  captured_paise  bigint NOT NULL DEFAULT 0,
  refunded_paise  bigint NOT NULL DEFAULT 0,
  mismatch        boolean NOT NULL DEFAULT false,
  version         bigint NOT NULL DEFAULT 0,
  updated_at      timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE webhook_events (                                  -- append-only source of truth
  event_id        text PRIMARY KEY,                            -- idempotency key
  order_id        text NOT NULL REFERENCES orders(order_id),
  type            text NOT NULL CHECK (type IN ('payment.failed','payment.authorized',
                       'payment.captured','refund.created','refund.processed')),
  amount_paise    bigint NOT NULL CHECK (amount_paise >= 0),
  api_version     text NOT NULL,
  sent_at         timestamptz NOT NULL,
  received_at     timestamptz NOT NULL DEFAULT now(),
  delivery_attempt int
);
CREATE INDEX ON webhook_events (order_id, sent_at, event_id);   -- the recompute path
CREATE TABLE processed_events (event_id text PRIMARY KEY, processed_at timestamptz DEFAULT now());
CREATE TABLE orphan_events   (like webhook_events, LIKE INCLUDING ALL);  -- reported, never applied
CREATE TABLE reconciliation_runs (id bigserial PRIMARY KEY, started_at timestamptz,
  cutoff_ist timestamptz, stats jsonb, status text);
```

- All money is `bigint` paise. **No floats anywhere** — ₹ conversion happens only in response renderers.
- `webhook_events` is the ledger; `orders` is a derived, recomputable projection. Monthly partitioning on `sent_at` once tables exceed ~200M rows; `orphan_events` kept 18 months for audit.

- Nightly job: rebuild any order from its log and diff against the projection — self-healing and audit-friendly.
- Reconciliation outputs (`order_states.csv`, `summary.json`) are produced by the same code path as the batch run in this repo — one truth, two entry points.

5. Failure handling & retries

FAILURE	BEHAVIOUR
Gateway → us (5xx/timeout)	Gateway retries with backoff (at-least-once) → our inbox dedupes. Receivers stay dumb & up.
Kafka down	Receivers buffer to local disk (spool) and shed load with 503 + `Retry-After`; alert fires at 30s.
Consumer crashes mid-order	Transaction rolls back; Kafka redelivers; idempotent reapply. No partial state.
Poison event (repeated crash)	After 5 deliveries → DLQ `gateway.events.dlq` + page; order flagged `NEEDS_REVIEW`; human replays after fix.
Refund/chargeback dispute	Operator re-runs nightly recompute for the order; ledger is the truth.
Bad deploy	Blue/green; consumers join group after passing shadow replay on 1% of traffic.

6. When the gateway ships API v3

- **Parse at the edge behind a version adapter**: `parse_event(raw) -> NormalizedEvent` with per-version implementations (`v1` ₹-string, `v2` paise-int, `v3` = whatever arrives). The state machine sees only normalized integer-paise events — **zero changes** below the parser seam.
- Contract tests in CI replay a captured real v3 sample corpus; schema registry (or JSON-schema files) with `additionalProperties: true` for tolerance.
- `api_version` allow-list lives in config, not code; unknown versions route to DLQ + alert not 500 (today: count as `invalid_events`).
- Dual-publish window: accept v2+v3 simultaneously; dashboards track traffic split; remove v2 only after 30 days of zero v2 volume.

7. How Finance knows when the numbers drift

- **Every event** emits metrics: `events_ingested_total{type,version}`, `duplicates_total`, `orphans_total`, `mismatches_total`, `consumer_lag`.
- **Per settlement cycle** (daily 02:00 IST job, after gateway file drops): compare our net settlement = Σ captured − 2% fees − Σ refunded vs the **gateway's settlement file**, paise-exact. Variance ≠ ₹0 → Sev-2 page to the on-call and a templated email to Finance with the order-level diff attached (`recon_diff_YYYY-MM-DD.csv`).
- **Drift detectors** (7-day rolling): orphan rate > 0.5%, duplicate rate > 2%, mismatch rate vs 30-day baseline ×3, fee variance vs 2% expectation, refund-without-capture count. Any breach → alert + Grafana annotation.
- **Weekly Finance email**: orders by state, captured vs settled vs refunded, top-10 mismatch orders with links, trend vs prior week — generated from `summary.json` by `scripts/generate_findings.py`, so the report and the books can never disagree.
- **Monthly access review** of the orphan report; true settlement sign-off is a human comparing three numbers: our summary, gateway statement, bank credit advice.

8. Capacity & cost sanity check

2,000 events/sec ≈ 173M/day ≈ 5B/month. At ~250 bytes/event the Kafka topic is ~45 GB/month; Postgres ledger ~1.5 TB/year with indexes on 16 TB NVMe — fine on one well-tuned primary + one replica. The batch reconciler in this repo processed 520k events in ~11 s at ~20 MB RSS — the hot path is I/O-bound and partitions linearly; the streaming design is deliberately the same algorithm the batch uses, so a bug fixes everywhere at once.

Bottom line: verify, dedupe, partition by order, recompute from the ledger — then measure settlement against the gateway every single day, and page a human the moment paise drift.